Task 3: Secure Coding Review

Language: Python 3 | **Application:** Flask web app (registration, login, greeting, ping, file download, data/config loading) | **File audited:** `vulnerable_app.py` (~80 lines)

1. Scope and Method

Item	Detail
Review type	White-box source code review
Manual inspection	Line-by-line review of every route: trace user input (`request.form`, `request.args`, `request.data`) to dangerous sinks (SQL, shell, templates, file system, deserializers)
Static analysis	Custom AST-based scanner (`mini_scan.py`) with rules modelled on Bandit/Semgrep checks. Bandit itself could not be installed (no network in this environment), so the same checks were reproduced by a small stdlib script. In your own setup you can run: `pip install bandit semgrep` then `bandit -r .` and `semgrep --config auto .`
Reference standards	OWASP Top 10 (2021), CWE, OWASP ASVS

2. Summary of Findings

Static scan result: **14 findings** on the original code (9 High-severity issues, plus Medium/Info). After fixes: **0 real findings** (3 flagged lines are false positives, see section 5).

#	Vulnerability	Location	Severity	CWE	OWASP 2021
1	SQL Injection (login)	`/login`, line 43	Critical	CWE-89	A03 Injection
2	SQL Injection (register)	`/register`, line 33	Critical	CWE-89	A03 Injection
3	OS Command Injection	`/ping`, line 57	Critical	CWE-78	A03 Injection
4	Insecure Deserialization (pickle)	`/load`, line 68	Critical	CWE-502	A08 Integrity Failures
5	Unsafe YAML load	`/config`, line 74	High	CWE-502	A08 Integrity Failures
6	Server-Side Template Injection / XSS	`/hello`, line 51	High	CWE-1336 / CWE-79	A03 Injection
7	Path Traversal	`/download`, line 63	High	CWE-22	A01 Broken Access Control
8	Hard-coded secrets	lines 12, 14	High	CWE-798	A07 Auth Failures
9	Weak password hashing (MD5, unsalted)	lines 31, 41	High	CWE-327 / CWE-916	A02 Cryptographic Failures
10	Debug mode enabled (Werkzeug console RCE)	line 80	High	CWE-489	A05 Misconfiguration
11	Bound to all interfaces	line 80	Medium	CWE-668	A05 Misconfiguration
12	No input validation / no	`/register`	Medium	CWE-20	A04 Insecure Design

#	Vulnerability	Location	Severity	CWE	OWASP 2021
	password policy				
13	Verbose "Welcome <username>" and unescaped output	`/login`	Medium	CWE-79	A03 Injection
14	No rate limiting / lockout on login	`/login`	Medium	CWE-307	A07 Auth Failures

3. Detailed Findings and Remediation

3.1 SQL Injection: CWE-89 (Critical)

Vulnerable code

```
query = f"SELECT * FROM users WHERE username='{username}' AND
password='{password}'"
user = db.execute(query).fetchone()
```

Attack: username `admin' --` makes the query `... WHERE username='admin' --' AND ...`, which skips the password check and logs in as admin.

Fix: parameterized queries

```
row = db.execute("SELECT password FROM users WHERE username =
?", (username,)).fetchone()
```

The same fix applies to the `INSERT` in `/register`. Also fetch by username only and verify the password hash in Python.

3.2 OS Command Injection: CWE-78 (Critical)

Vulnerable code

```
subprocess.check_output("ping -c 1 " + host, shell=True)
```

Attack: `/ping?host=8.8.8.8;cat /etc/passwd` runs an arbitrary command on the server.
Fix: no shell, argument list, allow-list validation, timeout

```
ipaddress.ip_address(host)   # raises ValueError if not a valid
IP
subprocess.run(["ping", "-c", "1", host], capture_output=True,
text=True, timeout=5)
```

3.3 Insecure Deserialization: CWE-502 (Critical)

Vulnerable code

```
data = pickle.loads(request.data)
```

`pickle` executes code during loading, so a crafted payload gives remote code execution.
Fix: use a data-only format (`json.loads`). If binary objects are unavoidable, sign the data with HMAC and verify before loading.

3.4 Unsafe YAML loading: CWE-502 (High)

Vulnerable: `yaml.load(data, Loader=yaml.Loader)` can instantiate arbitrary Python objects.
Fix: `yaml.safe_load(data)` .

3.5 Server-Side Template Injection / XSS: CWE-1336, CWE-79 (High)

Vulnerable code

```
render_template_string("<h1>Hello " + name + "</h1>")
```

Attack: `/hello?name={{7*7}}` prints `49` (template code is evaluated); more advanced payloads reach RCE. `<script>` input gives reflected XSS.
Fix: user input is data, never part of the template

```
render_template_string("<h1>Hello {{ name }}</h1>", name=name)
# Jinja2 auto-escapes
```

3.6 Path Traversal: CWE-22 (High)

Vulnerable code: `send_file(os.path.join("uploads", filename))`
Attack: `/download?file=../../etc/passwd`.
Fix: resolve the path and confirm it stays inside the upload folder

```
target = (UPLOAD_DIR / filename).resolve()
if UPLOAD_DIR not in target.parents or not target.is_file():
    abort(404)
return send_from_directory(UPLOAD_DIR, target.name)
```

3.7 Hard-coded Secrets: CWE-798 (High)

Vulnerable: `app.secret_key = "supersecret123"` and `ADMIN_PASSWORD = "admin@123"` in source code (visible in Git history forever).
Fix: load from environment variables or a secrets manager; generate with `secrets.token_hex(32)`; rotate any secret that was ever committed; add `.env` to `.gitignore`.

3.8 Weak Password Storage: CWE-327 / CWE-916 (High)

Vulnerable: `hashlib.md5(password.encode())`. MD5 is fast and unsalted, so rainbow tables and GPUs crack it in seconds.
Fix: a slow, salted password hash.

```
from werkzeug.security import generate_password_hash,
check_password_hash
hashed = generate_password_hash(password)
check_password_hash(stored_hash, password)
```

For new systems, Argon2id (`argon2-cffi`) or bcrypt are also good choices.

3.9 Insecure Configuration: CWE-489, CWE-668 (High/Medium)

Vulnerable: `app.run(host="0.0.0.0", debug=True)` exposes the Werkzeug interactive debugger (code execution) to the whole network.

Fix: `debug=False`, bind to `127.0.0.1`, and run in production behind Gunicorn/uWSGI plus a reverse proxy (Nginx) with HTTPS.

3.10 Missing validation, generic errors, brute-force protection (Medium)

- Validate length and format of all input (username 3 to 32 chars, password at least 10 chars).
- Return a generic `Invalid login` with HTTP 401 (never reveal whether the username exists).
- Add rate limiting (`Flask-Limiter`), account lockout or backoff, and optionally MFA.

4. Before / After Static Scan

Version	Findings
`vulnerable_app.py`	14 (9 High-risk categories: SQLi x2, shell injection, pickle, YAML, SSTI, hard-coded secrets x2, debug mode)
`secure_app.py`	3 flagged lines, all false positives (see 5)

Raw outputs: `scan_vulnerable.txt`, `scan_secure.txt`.

5. False Positives (Triage Notes)

Static tools over-report; each hit must be triaged manually:

- `render_template_string` at lines 53 and 60 of `secure_app.py`: the template is a **constant string** and user data is passed as a variable, so Jinja2 escapes it. Not exploitable.
- `subprocess.run` at line 70: uses an **argument list** (no shell) with an IP validated by `ipaddress`, so not injectable.

6. Secure Coding Best Practices (Checklist)

1. **Validate all input** with allow-lists (type, length, format); treat every request field as untrusted.

2. **Use parameterized queries / an ORM** (SQLAlchemy); never build SQL with string formatting.
3. **Avoid shells**: pass arguments as a list, never `shell=True` with user data. Avoid `eval`, `exec`, `os.system`.
4. **Encode output** for its context (HTML, JS, URL); keep template auto-escaping on.
5. **Use safe deserializers** (`json`, `yaml.safe_load`); never unpickle untrusted data.
6. **Hash passwords** with Argon2id / bcrypt / scrypt, salted; enforce a password policy and MFA.
7. **Manage secrets** outside code (env vars, Vault, cloud secret managers); rotate on exposure.
8. **Least privilege**: DB accounts with minimal rights; run the app as a non-root user.
9. **Secure configuration**: debug off, HTTPS/TLS everywhere, security headers (CSP, HSTS, X-Content-Type-Options), secure + HttpOnly + SameSite cookies, CSRF protection (Flask-WTF).
10. **File handling**: canonicalize paths, restrict to a base directory, validate upload type/size.
11. **Error handling and logging**: generic messages to users, detailed logs server-side, no secrets or passwords in logs.
12. **Dependency hygiene**: pin versions, scan with `pip-audit` / Dependabot.
13. **Automate**: run Bandit, Semgrep and `pip-audit` in CI on every pull request; add peer review and security unit tests.

7. Remediation Plan (Priority Order)

Priority	Action	Effort
P0	Fix SQL injection, command injection, pickle/YAML deserialization	Low
P0	Remove hard-coded secrets, rotate them	Low
P1	Fix SSTI/XSS, path traversal, switch to salted password hashing	Low-Medium

Priority	Action	Effort
P1	Disable debug, restrict binding, deploy behind HTTPS reverse proxy	Low
P2	Add input validation, rate limiting, CSRF, security headers	Medium
P3	Add Bandit/Semgrep/pip-audit to CI pipeline	Low

8. Conclusion

The original application contained multiple critical flaws that allow authentication bypass, remote code execution and file disclosure. Most root causes are the same pattern: **untrusted input reaching a dangerous sink without validation**. The remediated version (`secure_app.py`) removes every confirmed issue using parameterized queries, safe APIs, strong hashing, externalized secrets and secure defaults. Continuous static analysis plus manual review should be kept as part of the development workflow.

9. Deliverables

- `vulnerable_app.py` : audited code
- `secure_app.py` : remediated code
- `mini_scan.py` : static analyzer used
- `scan_vulnerable.txt` / `scan_secure.txt` : scan outputs
- `Secure_Coding_Review_Report.md` : this report